

PAGE 1 — Retrieval-Augmented Generation: Introduction

Retrieval-augmented generation, or RAG, is a design pattern that couples a language model with a retrieval system. Instead of asking the model to answer from its own parameters alone, the system first searches a knowledge base for the relevant passages and then hands those passages to the model as grounding context.

The motivating problem is hallucination. A model that answers from memory will happily produce fluent but factually wrong statements when it is uncertain. If the answer must instead be derived from retrieved evidence, the model is constrained to say only what the evidence supports.

A RAG pipeline has four stages: chunking the source documents, embedding each chunk into a vector space, searching that space for chunks similar to the question, and generating an answer from the retrieved chunks with citations.

The key difference from a naive chat wrapper is that retrieval is explicit and inspectable. Every claim in the final answer should trace back to a specific chunk, and when no chunk supports an answer, the system must say it does not know rather than inventing one.

This document is part of a portfolio project called Docent, which demonstrates each stage of the pipeline in production-quality detail. Page one covers the overall architecture; later pages cover chunking, embeddings, and retrieval quality.

PAGE 2 — Chunking Strategy

Chunking is the first and most underrated decision in a RAG system. The chunks are the atomic units of retrieval: a search can only return an entire chunk, so chunk size directly controls the granularity of the answers a user gets.

If chunks are too large, they contain too much unrelated material, and the embedding of the whole chunk dilutes the relevant passage; retrieval matches the general topic but the model has to wade through noise. If chunks are too small, individual facts are cleanly embedded but context is lost across the cuts.

Docent targets roughly five hundred tokens per chunk. That is long enough to carry a self-contained idea, yet short enough that a chunk stays topically coherent for embedding purposes.

Overlap matters just as much as size. If a single fact happens to straddle the exact boundary between two chunks, it is cut in half and neither half embeds into anything useful. Docent re-appends the final fifty tokens of every chunk onto the start of the next one, so the boundary region exists in both chunks and the fact stays retrievable.

Chunks are tracked with the page they came from. Page numbers are stored per chunk so that citations in the final answer can point the user to the physical location in the source PDF.

PAGE 3 — Embeddings and Vector Search

Embeddings map text into a high-dimensional vector space where semantically similar passages sit close together. Docent uses the OpenAI text-embedding-3-small model, which produces 1536-dimensional vectors and is cheap to run at scale.

Similarity between a question and a chunk is measured with cosine similarity: the dot product of the two unit vectors. This is a purely geometric notion of relatedness and requires no model inference at query time, which is what makes vector search fast.

Vectors are stored in Supabase Postgres using the pgvector extension. pgvector provides the vector column type and, crucially, an HNSW index that makes approximate nearest-neighbour search feasible even with millions of chunks.

Keeping vectors in the same Postgres database as the rest of the metadata means there is no separate vector database to operate. One database to reason about, one backup story, and no cross-service consistency problems.

Storing the embedding column and the human-readable content column side by side means a retrieval query can return the matched chunk text directly, without a second lookup round-trip to another store.

PAGE 4 — Citations and Hallucination Guardrails

Retrieval quality decides whether a RAG system can be trusted. Docent runs a re-ranking pass over the candidate chunks so the most on-point passages surface first, and it gates the answer on how well the evidence supports the question.

Every answer is generated with the retrieved chunks in the prompt and an instruction to cite the chunk identifiers it draws on. The UI renders those identifiers as inline links back to the source page in the PDF.

The visible confidence signal is a guardrail against fabrication. When the top retrieved chunks are not similar enough to the question, the system is configured to respond with an explicit refusal, such as "I could not find information about this in the uploaded documents", rather than guessing.

Declining to answer is a feature, not a failure. It is the honest behavior that separates retrieval-augmented generation from an unfaithful chatbot, and it is demonstrated on purpose in the Docent demo.

Known limitations in version one: images and tables inside PDFs are not extracted, and there is no multi-user authentication. Both are documented trade-offs for a focused portfolio demonstration of the retrieval pipeline itself.